

An Analysis of a Linux Based Operating System for Focused Childhood Education

Christopher Lozinski

School of Computer Science, University of Guelph

CIS*4910: Computer Science Thesis S26

Professor Stefan Kremer

12 August 2026

Table of Contents

Abstract	3
Technology use in the classroom.....	4
How do we define attention and distraction?	5
Problems with technology in the classroom.....	5
Methods of preventing digital distraction.....	6
Thesis questions	7
Project Design Methodology.....	7
Project Overview	7
Preventing Off Task Work with Launcher and RBAC	8
Preventing Multitasking Through Controlled Window Management.....	9
Makecode for Student Accessibility.....	10
Design Breakdown of Lesson Builder.....	10
Design Breakdown of Network Architecture.....	12
Analysis of Student, Teacher Interactions	13
Further Research and Project Expansion Opportunities.....	15
Bibliography	16

Abstract

Building on the functional Education focused Debian based Linux OS from my CIS*4900 project in W26, this thesis course intends to further the development of this system, from a minimum viable product / prototype to a scalable, research validated, classroom tool. In contrast to the CIS*4900 project, which focused on the creation of GUI elements and designing many different distinct elements of the system, this work is focused on two key features: the development of a flexible Lesson Builder GUI applet for teachers and educators to implement their own lesson designs into the system without prior technical experience; and the implementation of a secure, robust mDNS based networking architecture and interface to allow for easy, school wide device communication through local internet rather than a cloud provider. This report is intended to investigate the origins of digital distraction in the classroom and effects of working and studying within a locked down system. A review of the literature, an evaluation of how certain constrained interfaces impact cognitive load, student focus and the tendency for system breaking behaviour, alongside a technical breakdown of the finalized features will be presented.

1. Technology use in the classroom

Today, rather than pencils and notebooks, digital technologies have become the default tool of students for learning and completing work. Many students rely solely on laptops and tablets for learning. (Aagaard, 2015, p. 92) Applications such as Google Docs or Microsoft Word have been used for note taking and writing, Powerpoint for presentations, and Cloud service providers such as Dropbox and Google Drive for sharing files and collaborating on documents (Aagaard, 2015, p. 92-93) Students utilize search engines and sites such as wikipedia to check and challenge lesson materials. (Aagaard, 2015, p. 92-93) Additionally, the most recent technological developments such as Large Language Model AI, introduces new dynamics to the learning process, like 24/7 support with writing and answers to complex academic questions. (Martin et al., 2025, p. 3424) With all of these options, it can be overwhelming for students, especially younger children, to remain on task. Struggles with attention and distraction are extremely common in classrooms and have only been exacerbated by the introduction of unrestricted technologies. However, misunderstanding on how students learn, focus and become distracted have caused the technologies used in the classroom to retain little accountability for the distraction. Rather, the burden most usually lies with the student themselves and their perceived ability to control their focus. But is this entirely the case, or does the technology itself deserve to share part of the blame?

How do we define attention and distraction?

When a student uses a digital tool for education, often they also experience digital distraction: when a learner loses their academic focus by engaging with non academic material on their digital devices. (Martin et al., 2025, p. 3424) When a person is paying attention to an event or task, they are perceiving the discrete bits of information that exist in their external environment and processing them in their mind. (Aagaard, 2015, p. 91) Many students, when interacting with educational technologies such as laptops and tablets, experience an almost trance-like state that pulls them into distraction. If a student recognizes such attraction, they are able to break out of it and resist, but not without substantial time and energy. (Aagaard, 2015, p. 93-94) According to Aagaard, this phenomenon is known as habitual distraction; when a user becomes thoroughly familiar with their device over long periods of time, they form similarly ingrained habits. (p. 95) Such prereflective attraction towards frequently

visited pages causes the user to subconsciously act distracted, being unaware that they have gone off task at all during the process. (Aagaard, 2015, p. 94) These laptops and devices, being such attractive agents, make them active players in the distraction process rather than a neutral tool in the matter. The responsibility for a distraction thus is shifted from being solely that of the student to also partially to that of the technology itself, as it embodies the essence of the distraction. (Aagaard, 2015, p. 95)

Problems with technology in the classroom

As many students rely solely on laptops and tablets for learning, as opposed to physical materials, some educators may view this shift as the expansion and democratization of knowledge, and arguably so. However, this digital migration introduces its own set of problems to the learning experience. Over several interviews, students admitted to becoming frequently distracted on technologies during lessons or work sessions, with such behaviour normalized within educational contexts. (Aagaard, 2015, p. 93) Additionally, in the case of tablet based learning for younger students, the applications deployed have been shown to be, deliberately and inherently, highly interactive. At face value, while such applications appear to effectively direct attention towards learning content, they have created significant challenges for children's finite working memory. (McEwen & Dubé, 2015, p. 10) Presenting students, specifically youth, with complex applications for learning can have a harmful cognitive effect on students with lower executive functioning. (McEwen & Dubé, 2015, p. 18, 21) Such students have their attention so taken by the complexity of their tasks that they are unable to process helpful external information that students with higher executive function are able to utilize. Additionally, the normalization of multitasking and task switching within educational technology, the rapid and inefficient movement between tasks, leads to students carrying high cognitive costs. (Schmidt, 2020, p. 280) The brain is not able to process two complex streams of information at one time; learning tasks that require, for example, switching between multiple tabs and programs, leave students with severe mental fatigue and a comprehension of the material that is far shallower and less transferable to new contexts than if the learning had been simple and focused. (Schmidt, 2020, p. 280)

A systematic review of digital distractions in education by Martin et al. found that the most prevalent digital distractions were that of technology itself. Technological distractors accounted for 51.95% of all causes; alerts, ringtones, and the need to stay constantly

connected among other reasons. This was followed by distractions caused by personal needs (37.66%) (which includes entertainment like online games, shopping, etc), and instructional environmental factors, like engagement and poor classroom management, which sat at 10.39%. They also identified that digital distractions caused qualitatively visible consequences such as personal academic performance issues (at 66.67%), disrupted classroom instruction (23.33%) and broader problematic technology use (10%).

Methods of preventing digital distraction

With this, the question of how to improve the relationship between students and their technology must be considered. Many strategies for preventing digital distraction within the literature on childhood focus have been presented, to varying degrees of success. In the same review by Martin et al., they identified and classified three distinct categories of strategies that have been employed within the classroom to prevent distraction. (p. 3438) The first, classroom environment regulations (41.03%) regulated and modified the classroom environment itself to reduce digital distractions. (p. 3441) Of the studies that they reviewed, several found that effective management techniques, active instructional strategies and structured learning time all were suggested to minimise off task behaviour, along with collaborative peer instructions. (Martin et al., 2025, p. 3438) Strategies to directly control the use of digital devices (30.77% usage), like banning devices from the classroom or limiting the use of certain devices with more educationally focused tools were suggested to have varying degrees of success. (p. 3441) Finally at 28.21%, strategies that looked to intervene in personal behaviour and enhance self-regulation have also been used to prevent or reduce digital distraction. (Martin et al., 2025, p. 3441)

Another key recommendation for reducing digital distraction was for educational application designers themselves. To avoid disengaging lower EF students, it was suggested that developers should avoid app testing solely based on age or grade level, but rather should consider the diverse set of EF skills so that their apps design is applicable to all users. (McEwen and Dubé, 2015, p. 19-20) In the same study by McEwen and Dubé, an important distinction was made; children with lower EF reported enjoying simple applications significantly more than complex ones, while high EF showed no preference for either. (p. 19-20)

Additionally, a frequently cited reason for students becoming distracted was boredom and disengagement with lesson material. (Martin et al., 2025, p. 3438-3443) When a lesson is

not engaging, commonly students revert to the subconscious and reflective urge of habitual distraction on their device. By working to increase direct student engagement with materials and decreasing lulls in teaching, students are far less likely to become distracted and are able to retain the materials being taught. (Schmidt, S. J., 2020, p.285)

2. Thesis Questions

The central hypothesis of this project is that, based on the sources provided, standard operating systems are by their nature, overly complex and ill equipped for classroom instruction. This is because they provide students with unrestricted access to a variety of different softwares, and the internet, which introduces opportunities for habitual distraction. As such, this project and thesis report is looking to address the following two questions:

Q1: How does a constrained, role-based desktop environment affect student focus and task completion when compared to standard operating systems and applications for learning?

Q2: How does such a constrained environment positively or negatively impact teacher and student cognitive load and engagement with material?

3. Project Design Methodology

Project Overview

This project is an educational operating system built on a minimal Debian Linux base, debootstrap, intended to provide a distraction-free, locked down environment for primary and secondary school student learning of mainly programming and STEM concepts. During the project development phase, in the course CIS*4900, the focus was on building out a functional, minimum viable product that showcased all of the essential GUI elements. The scope of this course, CIS*4910, was to refine what was developed over the span of CIS*4900 and create two key features:

1. A lesson builder application for the teacher user, where they are able to customize individual tutorial lessons for their classroom, and,
2. A revised network architecture and interface for classroom connectivity and file sharing, where students are able to connect to their teachers device without the use of internet or cloud providers, if desired.

These two features worked to demonstrate how the concept underlying this system would be applied and used within a classroom setting. In addition to these, the other central application integrated into the system is Microsoft's MakeCode Arcade, an open-source block code editor that allows students to build games and interactive programs using visual blocks which can be translated into JavaScript or Python code. To effectively address and prevent digital distraction from occurring within this operating system, several design and system choices were based on the research on attention and distraction.

Preventing Off Task Work with Launcher and RBAC

For permissions and file access, this operating system implements role based access control. Role based access control (RBAC) is a system structure model where end user access to systems, applications and data is determined based on a user's predefined role within the system. (Clark. B, 2026) Through the restriction of user access to resources, traditionally, RBAC is meant to simplify access management and defend against malicious user attacks. (Clark. B, 2026) Within this system, however, the implementation of RBAC is intended to:

- reduce and prevent off task work that has been proven to cause distraction for students, and,
- prevent more technically proficient student users from bypassing such restrictions to go off task within the system.

In this project, role based access control uses a two role model:

- Teacher user: granted full administrative access to configure classrooms, assign lessons, and manage resources,
- Student user: entire application environment is decided by the teacher in what class they are currently in, (makes local connection to teacher device) by the teacher in real time, has no access to modifying system resource allocation, file structure, or application assignment. The student user operates only within their predetermined profile of selected apps for their specific classroom at any said time.

This structure is enforced at the operating system level through Linux user groups, file permissions, and network state synchronization. In order to accommodate the differences in application permissions between users, the primary navigation interface is a vertical sidebar app launcher. The launcher, based on the teachers permission decisions and classroom information, decides which apps and files appear for a respective student at runtime, other

than a simple pre-enabled “Join Classroom” application. This way, students will only have access to a specific set of applications enabled and chosen by their teacher depending on their classroom and current lesson. For the teachers, their sidebar contains a fixed list of distinct applications to allow for flexible and adaptable teaching structures.

Preventing Multitasking Through Controlled Window Management

In order to reduce the cognitive load of students of all levels of executive functioning, care was taken when designing the functionality of window management within the user interface. To control the placement and appearance of app windows on the screen, the system employs i3, a common Linux, tiling window manager. The sidebar itself is a Python application, which is placed to the left of the screen by i3 on startup, within a container. All other windows that open on the system, open to the right of the sidebar, within their own container. The i3 window manager offers two different types of window organisations: tiled, where windows open next to each other vertically or stacked, and tabbed, where windows stack on top of each other, with one visible at a time, switching focus through the use of browser-like tabs.

This system employs the use of both types of systems in order to provide the most focused experience for the user. On start-up, the two containers, the sidebar and space for applications, are tiled next to each other, the sidebar taking 11% of the screen space and app window taking the remaining 89%, which is set by a layout JSON file, loaded on start. Then, inside the application container, a child container is created that is marked as tabbed. All applications when opened, open within this child container as a tab, which is not visible to any user. When an application button is pressed on the sidebar, i3 switches focus to the app container child, and opens a new tab within or, if already open, switches back to the requested app, with only a single application being allowed to run and appear on the screen at a time. This removes the ability for split screen multitasking while still allowing for multiple application work if needed, enforcing a focused, simple, design. As well, the sidebar cannot be closed or altered in size, outside of the minimize button, by the user which ensures that students always have a navigation path back to any of their applications.

Makecode for Student Accessibility

Microsoft's Makecode was chosen as the core educational application of the system. As mentioned, it is a primarily block-based, but can be translated into text-based (Python and Javascript) coding environment for building retro-style arcade games. It can also be used to teach programming fundamentals through guided tutorials and skillmaps. The system runs a complete offline copy of the editor, served locally over HTTP and rendered inside a GTK window using WebKit2. No internet connection is required after installation. Makecode has a built in pixel art editor, bit music maker and has options that walks the student step by step through the lesson process, as well as having extensive documentation for education. One of the primary reasons why Makecode was chosen as an application was its simple yet stimulating visuals and lessons. Simple applications with a single learning mechanic and plain visuals were shown to be the most effective and directing a child's attention towards content that not only was essential for completion of the task assigned to them but also for having an applied understanding. (McEwen and Dubé, 2015, p. 13)

The choice of Makecode was made in order to reduce cognitive load and increase concept retention, as children with higher EF had no hindrance to their learning when using simple applications, as opposed to lower EF children, who suffered negative cognitive effects through work in complex applications. (McEwen and Dubé, 2015, p. 21) Using the "Lesson Config" application built into the system, teachers can enable or disable specific student access to lessons and documentation that appear within the student wrapper interface for Makecode. This maintains that the student does not become distracted from what is being requested of them. As well, serving Makecode as a static, offline website, rather than one over the internet, means a student does not feel the allure of the internet affecting their subconscious during their focused work. (Aagaard, 2015, p. 93-95)

Design Breakdown of Lesson Builder

The lesson builder application is a teacher only GTK (Gimp Took Kit) app intended to be used for the creation of step by step MakeCode tutorials, which can then be published to students within a particular classroom. The python application first starts a background HTTP server and opens one single GTK window that contains two children, similar to how i3 organizes the sidebar and application view. Using "GTK paned," the window is split into two

independent WebKit view instances, the left side having the teacher control interface, with lessons, step numbers and descriptions, and the right containing an iframe with the live MakeCode webview, shown in Figure 1.

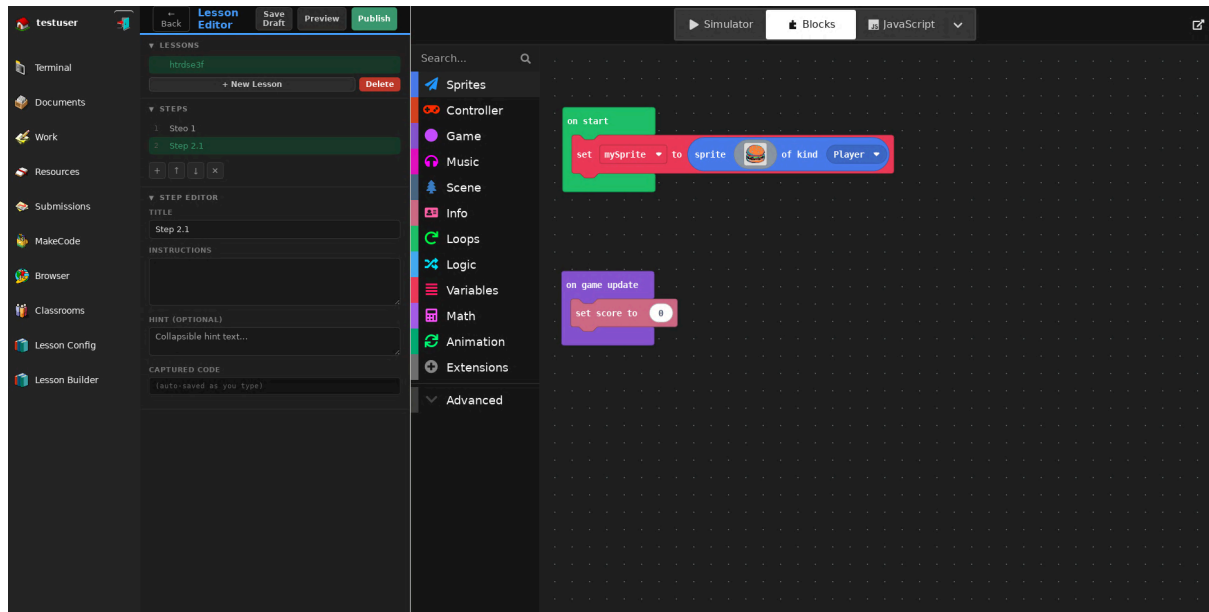


Fig. 1: interface of lesson builder application, showing the two different containers

As the teacher adds code to the Makecode window, it saves into their currently selected step within their interface, to a TypeScript file and XML file, that contains the blocks themselves. The teacher can add steps and switch between steps that contain different code, with the MakeCode webview refreshing and updating on each switch based on what is stored within lesson storage per step rather than within MakeCode itself. When the teacher wishes to publish the lesson to their classroom, three files are written, the meta information as a JSON, the full solution as a type script file and a markdown “tutorial.md” file generated by a convert lesson program that renders the dictionary into the markdown format readable by MakeCode tutorial feature. Students are able to go through tutorials at their own pace, moving onto the next step once the required code from the previous step has been completed. On the completion of a tutorial lesson, students are granted access to the rest of MakeCode’s coding features that were hidden during the tutorial session as they were not necessary for the completion of the lesson. This allows for students with higher executive function, who may complete the lesson early, to build on their work and engage with more challenging material on their own. As referenced previously, a key cause of distraction was lack of student engagement with materials and boredom; when something becomes boring or unengaging,

students revert to their subconscious, reflexive urge for distraction on their device. By allowing for direct student engagement and expansion of lesson material once completed, boredom and lulls within the lesson become less common. This benefits student enjoyment and increases learning comprehension of materials by rewarding creativity and application, while providing ample time for teachers to work with students who may struggle more with certain concepts or materials.

Design Breakdown of Network Architecture

On login of a student machine, a student account has its own network.agent, student0agent@<user>.service, auto run by i3 for the role. The teacher has a long lived process, network.publisher, run during its session, which is a systemd unit, teacher-publisher.service that has the user as its root. In order for the student service to find the teacher's machine on the network it goes through a series of checks to try and pair. First, it checks to see if there is a manually configured address, then it looks for the last successful address it connected to and tries to confirm whether the teacher machine is still reachable at said address. Lastly, there is a discovery service if both other options fail that uses Multicast DNS (mDNS). mDNS is a zero configuration protocol that allows for devices to resolve hostnames to IP addresses without a dedicated DNS service. The student agent searches the local network for the teacher service using this protocol, while the publisher broadcasts a message on a UDP port every few seconds as a fallback. Since the student agent attempts to reconnect to its last known address, a student is able to easily and quickly reconnect on every login to a classroom that they have joined previously rather than having to research the entire network again. In order for a student to join a classroom in the first place, they use the Join Classroom application enabled by default on their machine, where they can enter a random hash code provided to them by their teacher.

The Classroom Manager application is used by the teacher user to add, remove and organize which students are in what classroom and what applications are available to the students enrolled within it. Within a classroom selected in this application, a teacher opens the classroom for joining, which presents the six character randomized code, which rolls over every five minutes. When a student enters this code into their computer, it sends it to the teacher's publisher over the network. If it matches and has not expired, then the publisher enrolls the student into the classroom roster and sends back a unique key for that classroom to

the student machine. The student agent saves and uses this code as it runs in the background as validation for all future connections to prove that they are enrolled in this specific classroom. Once pairing has completed, the agent and publisher exchange a set of messages each time the student machine is online and logged in containing its currently enabled applications. The publisher then compares this against the classroom's current settings and either acknowledges that nothing has changed or sends an updated configuration of lessons and applications, signed with the key, that is then refreshed on the student machine. The design of the classroom network being able to run everything locally, disconnected from the internet, was intentionally chosen to prevent distraction from occurring. Having a process where the students must go to a third party website or application introduces opportunities for distractions caused by the subconscious allure of the internet. Ensuring that the internet is not a prerequisite to network connectivity means this problem does not exist for users of the system, reducing opportunities for distraction to be introduced. As well, it means that the system is able to function completely within environments, such as remote classrooms, that may not have access to the internet, increasing accessibility.

5. Analysis of Student, Teacher Interactions

When compared to the causes of digital distraction found within current available educational applications, this operating system, through the use of restraint, directly addresses the challenges identified within the literature reviewed in this paper. The primary challenge within a standard operating system used in a classroom environment is their multistability, where they are meant to meet the needs of all different types of users. (Aagaard, 2015, p. 91) While this allows them to be widely applicable to a variety of professional circumstances, it provides students with a backdoor to distraction in a classroom setting. (Aagaard, 2015, p. 93) Habitual distraction is a prereflective urge, not a conscious decision. (Aagaard, 2015, p. 95) Standard systems and applications rely solely on a student's self regulation and control to prevent habitual distraction, which is not only ineffective but not based on any understanding of the origin of distraction itself. As well, problematically, many policy makers treat the uneducated and simplistic notion that banning or confiscating devices during the school day, as a panacea to distracted learning. (Schmidt, 2020, p. 282) Such actions have been shown to lead to nomophobia in students, the anxiety of being disconnected. (Schmidt, 2020, p. 282) This makes the restricted device more powerfully attractive than it was before, while refusing

to acknowledge the responsibility technology has as a contributing agent to habitual distraction. (Aagaard, 2015, p. 95)

The design of this operating system was to instead center on understanding of Human Computer Interaction and childhood psychology. Each decision was intended to directly reshape the user's involvement with a device and shift the responsibility of focus back to the student from the system level, rather than merely through applications. Through the implementation of a tightly controlled model of RBAC, this system foundationally addresses the problems that emerge with undefined permissions to applications, allowing for student focus to be directed in full by a teacher to a specific task. This interrupts the subconscious nature of habitual distraction that relies on automatic movements to off task activities. (Aagaard, 2015, p. 95) Controlled application window management contributes to this interruption, as well as addressing the high cognitive cost of task switching. This protects the working memory of students during their work, repositioning the role of a teacher to being that of an educator instead of a warden. (Schmidt, 2020, p. 281-283)

Applications chosen to be integrated within this window also address these challenges, along with the identified problems with over complexity. Prioritising simplicity within the system design and through applications such as MakeCode means that learners with higher EF, who wish to expand on their learning, are granted self imposed challenges. As such, the causes of boredom and passivity that trigger habitual distraction to occur are replaced with challenges tailored to meet needs of students with a diverse set of EF, rather than solely being based on age or grade. (McEwen & Dubé, 2015, p. 21) These challenges, then, are more rewarding and engaging to students who choose to self impose them rather than feeling as if they are repetitive and mundane exercises to keep them occupied. (McEwen & Dubé, 2015, p. 21) In contrast, students with lower EF do not feel overloaded by the material and are able to take effective breaks if needed without falling behind, incentivizing and encouraging task completion. (McEwen & Dubé, 2015, p. 21) Housing all of these system features offline acknowledges the fact that over 50% of digital distractions are caused by traditional characteristics of technology, such as constant connectivity and notifications. (Martin et al., 2025, p. 3438) Rather than removing the device from the classroom altogether, the offline nature of this system eliminates the root cause itself.

In banning technology outright, educators choose to send a message to students that they are incapable of using technology appropriately and require policing, stripping them of any sense of self determination as rational agents. (Schmidt, 2020, p. 282) When surveyed,

students expressed a desire to be in charge of their own learning and treated with the same respect as adults, emphasising their role as consumers of a public service. (Schmidt, 2020, p. 283) Reconfiguring the relationship between the user and the device instead acknowledges the autonomy of the student, which increases their feeling of respect and dignity, and prevents the anxiety caused by disconnection. (Schmidt, 2020, p. 281)

6. Further Research and Project Expansion Opportunities

Opportunities for future expansion and research could lie in researching the effects of this system's use within an educational setting, conducting qualitative interviews with educators and testing usability with students. This would identify and document any barriers and oversights that exist within the current system that could impact the learning process for different types of students. In addition to this, research on the effect of the rise of Large Language Models on cognitive offloading and distraction in K-12 students could be valuable. It would provide a foundation on which specific applications and skills could be prioritized in the classroom to enforce critical thinking skills that does not discard the use of the technology outright.

Bibliography

- Aagaard, J. (2015). Drawn to distraction: A qualitative study of off-task use of educational technology. *Computers & Education*, 87, 90–97.
<https://doi.org/10.1016/j.compedu.2015.03.010>
- Clark, B. (2026). Role-based access control implementation guide. In *Ibm.com*.
<https://www.ibm.com/think/topics/role-based-access-control-implementation>
- Martin, F., Long, S., Haywood, K., & Xie, K. (2025). Digital distractions in education: A systematic review of research on causes, consequences and prevention strategies. *Educational Technology Research and Development*.
<https://doi.org/10.1007/s11423-025-10550-6>
- McEwen, R. N., & Dubé, A. (2015). Engaging or distracting: Children’s tablet computer use in education. *Scholaris.Ca*, 18(4).
<https://mcgill.scholaris.ca/items/909bd212-ddbb-432e-8272-79ef47cd29f5>
- Schmidt, S. J. (2020). Distracted learning: Big problem and golden opportunity. *Journal of Food Science Education*, 19(4), 278–291. <https://doi.org/10.1111/1541-4329.12206>